Window functions, polia, JSON

Ján Mazák

FMFI UK Bratislava

Window functions

- Použitie GROUP BY vedie k nahradeniu celej skupiny jediným riadkom.
- Niekedy však chceme zachovať pôvodné riadky a len k nim doplniť hodnotu súvisiacu s inými riadkami v skupine.
- Riešenie: window functions. Tieto funkcie vidia nielen jeden riadok, ale aj „okolité“ riadky (presná definícia je súčasťou dotazu v časti OVER).
- Window functions nevidia riadky odfiltrované vo WHERE a HAVING.
- Ak chceme filtrovať výsledok podľa vypočítanej hodnoty window funkcie, treba použiť vnorený dotaz.

Window functions

```
SELECT emp_id, name, dept_id, salary,
       avg(salary) OVER (PARTITION BY dept_id) avg_s
FROM employee;
```

```
 emp_id | name   | dept_id | salary  | avg_s
--------+--------+---------+---------+------
     11 | King   |      10 | 5000.00 | 2600.00
     13 | Clark  |      10 | 1500.00 | 2600.00
     24 | Miller |      10 | 1300.00 | 2600.00
     14 | Jones  |      20 | 2975.00 | 2175.00
     ...
```

Window functions

```
SELECT dept_id, name, salary,
       rank() OVER (PARTITION BY dept_id
                    ORDER BY salary DESC)
FROM employee;
```

```
 dept_id |  name   | salary   | rank
---------+---------+----------+-----
      10 | King    | 5000.00 |    1
      10 | Clark   | 1500.00 |    2
      10 | Miller | 1300.00 |    3
      20 | Ford    | 3000.00 |    1
      20 | Scott   | 3000.00 |    1
      20 | Jones   | 2975.00 |    3
      20 | Adams   | 1100.00 |    4
    ...
```

Window functions: dense_rank(), row_number()

```
SELECT name, salary, rank() OVER w,
       dense_rank() OVER w, row_number() OVER w
FROM employee
WINDOW w AS (PARTITION BY dept_id ORDER BY salary DESC);
```

```
  name   | salary   | rank | dense_rank | row_number
---------+----------+------+------------+-----------
 Ford    | 3000.00 |    1 |          1 |          1
 Scott   | 3000.00 |    1 |          1 |          2
 Jones   | 2975.00 |    3 |          2 |          3
 Adams   | 1100.00 |    4 |          3 |          4
 ...
```

- dense_rank(): po zhode nevynecháva čísla.
- row_number(): čísluje 1, 2, 3, ... aj pri zhode; poradie zhodných riadkov je ľubovoľné, deterministicky: ORDER BY salary DESC, emp_id.

Window functions

Funkcie, ktoré možno použiť ako window functions:

- bežné agregačné funkcie
  (môžu sa však správať trochu odlišne!)
- rank(), dense_rank(), row_number(), percent_rank()
- nth_value() — element in the n-th row of the window
- ...

Window functions

- Ak vynecháme PARTITION BY, partícia obsahuje všetky riadky výstupu.

```
SELECT salary, sum(salary) OVER ()
FROM employee;
```

Window functions

- Ak vynecháme PARTITION BY, partícia obsahuje všetky riadky výstupu.

```
SELECT salary, sum(salary) OVER ()
FROM employee;
```

- Okno možno pomenovať a použiť viackrát.

```
SELECT
    sum(salary) OVER w,
    avg(salary) OVER w
FROM employee
WINDOW w AS (PARTITION BY dept_id
             ORDER BY salary DESC);
```

Window frame

- Každý riadok má svoje vlastné okno (window frame) — zoznam riadkov, nad ktorými sa vykoná window function.
- Okno závisí od použitej funkcie. Default window frame možno v rámci dotazu modifikovať.
- Komplikovaná syntax. Nasledujúce dve formulácie sú ekvivalentné významovo, ale vraj nie z hľadiska efektívnosti výpočtu...

```
ROWS UNBOUNDED PRECEDING EXCLUDE CURRENT ROW
ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
```

Window frame

```
SELECT salary, sum(salary) OVER (ORDER BY salary)
FROM employee;
```

```
 salary  |   sum
---------+------
  800.00 |   800.00
  950.00 |  1750.00
 1100.00 |  2850.00
 1300.00 |  4150.00
 1500.00 |  7150.00   <-- pribudlo 3000, nie 1500
 1500.00 |  7150.00
 1750.00 |  8900.00
```

Okno: riadky v partícii po daný riadok plus riadky s rovnakou hodnotou.

Vzťah M:N — pomocná tabuľka

Zamestnanec môže pracovať na viacerých projektoch a na projekte môže pracovať viacero zamestnancov. Takýto vzťah nemožno uložiť v jednej hodnote:

- v riadku zamestnanca by bol zoznam projektov (pole),
- v riadku projektu by bol zoznam zamestnancov.

Preto vzťah ukladáme do pomocnej tabuľky `emp_projects`, kde je jeden riadok pre každú dvojicu (zamestnanec, projekt):

```
emp_id | proj_id
-------+--------
    15 |       1
    15 |       2
    17 |       1
```

Zamestnanec 15 pracuje na projektoch 1 a 2. Na projekte 1 pracujú zamestnanci 15 a 17.

Vzťah M:N — pomocná tabuľka

```
CREATE TABLE project (
    proj_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL
);

CREATE TABLE emp_projects (
    emp_id INTEGER NOT NULL,
    proj_id INTEGER NOT NULL
);
```

Join prebieha len cez rovnosť jednoduchých stĺpcov, takže sa dá indexovať a optimalizátor ho vie spracovať bežnými algoritmami pre join. Pri poli by tento join už nebol rovnosťou jednoduchých stĺpcov: museli by sme hľadať hodnotu medzi prvkami poľa.

Kedy má pole zmysel

Hoci zložené hodnoty vo všeobecnosti nepoužívame, pole sa hodí, keď ide o:

- hodnotu, ktorú prirodzene tvorí n-tica, napr. súradnice bodu (x, y, z),
- hodnoty rovnakého druhu, ktorých počet sa mení, a ktoré by inak zaberali veľa stĺpcov väčšinou s `NULL`, napr. namiesto stĺpcov `telefon1` až `telefon10`,
- podmnožinu konečného zoznamu hodnôt, ktorú čítame a meníme ako celok, napr. tagy knihy `'fantasy'`, `'horor'`, `'humor'`, ktoré nemajú vzťahy k iným tabuľkám,
- výstup, ktorý zostavujeme z dát uložených nekompozitne, napr. zoznam projektov každého zamestnanca.

Array datatype

Databázy bežne podporujú polia (nie však SQLite).
Dátový typ v PostgreSQL:

```
CREATE TABLE x (
    y  integer[4],
    z  text[][]
);
```

Zápis v súlade s SQL štandardom:

```
CREATE TABLE x (
    y  integer ARRAY[4],
    z  text ARRAY[][]
);
```

Aktuálna implementácia veľkosť poľa ignoruje.

Arrays

Pole možno vytvoriť priamo v rámci dotazu.

```
SELECT ARRAY[1,2] || ARRAY[3,4] AS a;
SELECT ARRAY[1, 2] || '{3, 4}';
```

Na prácu s poľom existuje množstvo funkcií, napr. `array_prepend`, `array_length`.

```
SELECT array_position(
    ARRAY['mon','tue','wed','thu','fri','sat','sun'],
    'mon'
  );
SELECT * FROM x WHERE 47 = ANY (some_array);
SELECT * FROM x WHERE 47 = ALL (some_array);
```

Arrays

Pole tiež možno získať aplikovaním agregačnej funkcie.

```
SELECT e.name, array_agg(p.name)
FROM employee e
    JOIN emp_projects ep ON ep.emp_id = e.emp_id
    JOIN project p ON p.proj_id = ep.proj_id
GROUP BY e.emp_id, e.name;
```

```
     name      |          array_agg
---------------+-----------------------------
 MoneyMagnet   | {Enviro1,Enviro2,Nuclear3}
 SalesManager  | {Enviro2,Nuclear3}
 InsightNinja  | {Nuclear3}
 MrFixIt       | {Enviro1}
 DealMaster    | {Enviro1}
```

Tento dotaz nezaručuje konkrétne poradie prvkov vo výsledných poliach.

JSON a jsonb

Čiastočne štruktúrované dáta: každý záznam môže mať iné atribúty.
PostgreSQL má typ `json` (text) a `jsonb` (binárna reprezentácia, indexovateľná).

```
ALTER TABLE employee ADD COLUMN contact jsonb;

INSERT INTO employee (emp_id, name, contact)
VALUES (11, 'King', '{"phone": "555-0101",
  "email": "king@example.com",
  "languages": ["sk", "en"]}');
```

JSON — prístup k hodnotám

- `->` a `->>`: prístup ku kľúču
- `contact->'phone'` vracia `jsonb`
- `contact->>'phone'` vracia text,
- `contact ? 'email'` overí, či kľúč existuje,
- `contact->'languages'->>0` je prvý prvok poľa.

```
SELECT name FROM employee
WHERE contact @> '{"email": "king@example.com"}';
```

Operátor @> (containment)

`a @> b` je pravda, ak `jsonb` hodnota `a` obsahuje všetko, čo je v `b` (kľúče aj hodnoty, aj vnorene).

```
'{"a": 1, "b": 2}' @> '{"a": 1}'      --- TRUE
'{"a": 1}' @> '{"a": 1, "b": 2}'      --- FALSE
'["sk", "en"]' @> '["en"]'            --- TRUE
'{"c": {"ip": 1, "x": 2}}' @> '{"c": {"ip": 1}}' --- TRUE
'{"c": {"ip": 1}}' @> '{"ip": 1}'     --- FALSE
```

Pole sa porovnáva po prvkoch, na ich poradí nezáleží; vnorený objekt sa porovnáva rekurzívne.

Príklad: logovanie pre web (REST API)

```
CREATE TABLE request_log (
  id SERIAL PRIMARY KEY,
  payload jsonb
);

INSERT INTO request_log (payload) VALUES
 ('{"method": "GET", "path": "/api/employees/11",
   "status": 200, "duration_ms": 12,
   "client": {"ip": "10.0.0.7"}}'),
 ('{"method": "GET", "path": "/api/projects",
   "status": 500, "duration_ms": 903,
   "client": {"ip": "10.0.0.7"}}');

SELECT payload->>'path', (payload->>'duration_ms')::int
FROM request_log
WHERE (payload->>'status')::int >= 500;

SELECT id, payload->'client'->>'ip'
FROM request_log
WHERE payload @> '{"client": {"ip": "10.0.0.7"}}';
```

json_extract — JSON v SQLite

SQLite nemá samostatný JSON dátový typ. JSON ukladáme ako text alebo (od verzie 3.45) vo formáte JSONB typu BLOB. Prístup k hodnotám: `json_extract`, `->`, `->>`.

```
SELECT name, json_extract(contact, '$.phone') AS phone
FROM employee;

SELECT name FROM employee
WHERE json_extract(contact, '$.phone') = '555-0101';

SELECT name, json_extract(contact, '$.languages[0]')
FROM employee
WHERE contact IS NOT NULL;

SELECT name,
       contact -> '$.phone' AS json_val,
       contact ->> '$.phone' AS sql_val
FROM employee
WHERE contact IS NOT NULL;
```

Pole, jsonb, alebo tabuľka?

- pole: krátky homogénny zoznam, ktorý čítame a meníme ako celok
- jsonb: rôznorodé voliteľné atribúty bez vzťahov k iným tabuľkám, namiesto mnohých stĺpcov s väčšinou NULL
- samostatná tabuľka: vzťahy s inými tabuľkami, cudzie kľúče, many-to-many relationships

Literatúra

- https://www.sqltutorial.org/sql-window-functions/
- https://www.sqlite.org/windowfunctions.html
- https://www.postgresql.org/docs/current/tutorial-window.html
- https://www.postgresql.org/docs/current/arrays.html